

Nick Le Large, Frank Bieder*, and Martin Lauer

Comparison of different SLAM approaches for a driverless race car

Ein Vergleich von unterschiedlichen Ansätzen zur simultanen Lokalisierung und Kartierung unter Verwendung eines fahrerlosen Rennfahrzeuges

<https://doi.org/10.1515/teme-2021-0004>

Received January 20, 2021; accepted February 28, 2021

Abstract: For the application of an automated, driverless race car, we aim to assure high map and localization quality for successful driving on previously unknown, narrow race tracks. To achieve this goal, it is essential to choose an algorithm that fulfills the requirements in terms of accuracy, computational resources and run time. We propose both a filter-based and a smoothing-based Simultaneous Localization and Mapping (SLAM) algorithm and evaluate them using real-world data collected by a Formula Student Driverless race car. The accuracy is measured by comparing the SLAM-generated map to a ground truth map which was acquired using high-precision Differential GPS (DGPS) measurements. The results of the evaluation show that both algorithms meet required time constraints thanks to a parallelized architecture, with GraphSLAM draining the computational resources much faster than Extended Kalman Filter (EKF) SLAM. However, the analysis of the maps generated by the algorithms shows that GraphSLAM outperforms EKF SLAM in terms of accuracy.

Keywords: Automated racing, formula student driverless, SLAM, localisation, mapping.

Zusammenfassung: Für den Einsatz eines automatisierten, fahrerlosen Rennwagens soll eine hohe Kartierungs- und Lokalisierungsqualität für das erfolgreiche Rennfahren auf bisher unbekannten, engen Strecken gewährleistet werden. Um dieses Ziel zu erreichen, ist es wichtig, einen Algorithmus zu wählen, der die Anforderungen in Bezug auf Genauigkeit, Rechenleistung und Laufzeit erfüllt. Wir schlagen sowohl einen filter- als auch einen glättungs-basierten SLAM-Algorithmus vor und evaluieren diese anhand realer Daten, welche von einem fahrerlosen Formu-

la Student-Rennwagen gemessen wurden. Die Genauigkeit wird durch den Vergleich der SLAM-generierten Karte mit einer durch hochpräzise DGPS-Messungen erfassten Karte ermittelt. Die Ergebnisse der Auswertung zeigen, dass beide Algorithmen die geforderten Zeitvorgaben dank einer parallelisierten Architektur einhalten, wobei GraphSLAM die Rechenressourcen deutlich stärker beansprucht als der EKF SLAM Algorithmus. Die Analyse der von den Algorithmen erzeugten Karten zeigt jedoch, dass GraphSLAM den EKF SLAM Algorithmus im Bezug auf die Genauigkeit übertrifft.

Schlagwörter: Automatisiertes Rennfahren, Formula Student Driverless, SLAM, Lokalisierung, Kartierung.

1 Introduction

When it comes to new technology in the automotive industry, racing has quite often played a key role in fueling innovation and pushing cars to the limit of what is possible. Technologies like all-wheel drive, semi-automatic gearboxes or materials like carbon fiber have made their way into production vehicles through their application in race cars. Racing series like Formula E [1] or Roborace [2] have been founded specifically with this in mind. Formula Student [3] is another competition with this purpose but the race cars are developed exclusively by students. The cars are competing in various, loosely connected events all over the world. Following the recent technological trends, Formula Student Driverless was recently introduced as a new competition. It challenges students all over the world to develop and build self-driving single-seater race cars. One of the greatest challenges of automated vehicles is the problem of "Simultaneous Localization and Mapping".

1.1 Problem statement

Localization and mapping can be one of the bottlenecks of the overall system. First of all, bad localization performance leads to inaccurate maps and vice versa [4]. But ad-

*Corresponding author: Frank Bieder, FZI Forschungszentrum Informatik, Haid-und-Neu-Straße 10-14, 76131 Karlsruhe, Germany, e-mail: bieder@fzi.de

Nick Le Large, Martin Lauer, Institut für Mess- und Regelungstechnik (MRT), Karlsruher Institut für Technologie (KIT), Engler-Bunte-Ring 21, 76131 Karlsruhe, Germany



Figure 1: Our measurement vehicle KIT19d, KA-RaceIng's 2019 driverless car (Photo: Amadeus Bramsiepe) and the layout of our test track (Photo: Google Earth).

ditionally, in autonomous racing, if the map quality is not sufficient, the trajectory planning needs to diverge from the ideal racing line in order to leave big enough safety margins to assure the vehicle will not leave the track. The controller highly depends on the quality of the localization because it will try to reduce the difference between the estimated state and the planned state. If the localization is not accurate enough, the controller will not be able to reduce the error and the vehicle might leave the track. In the worst case, the SLAM algorithm might even diverge, which may lead to an emergency break manoeuvre or an accident.

With this in mind, we aim to choose an algorithm, which is able to deliver high map and localization quality in the context of racing applications. This implies extreme requirements like retaining high accuracy to navigate on narrow tracks while having real-time constraints at high measurement rates. In addition, the algorithms have to operate using only the limited resource of a light-weight, single-seated race car.

1.2 Related work

Formula Student Driverless is a fairly new competition. However, as it is a sport performed exclusively by stu-

dents, there are a number of publications discussing various parts of the autonomous system of these race cars. Zeilinger et al. [5] and Valls et al. [6] present the overall autonomous system of their respective vehicles. Both teams utilize SLAM algorithms based on an EKF and while they give a good overview over the building blocks needed for autonomous racing, they only touch SLAM briefly and do not evaluate different approaches. Nekkah et al. present the software stack of the autonomous vehicle that this work is based on [7]. Their SLAM algorithm serves as a base line for the experimental evaluations made in Section 4.

SLAM evaluations have been performed in the context of Formula Student Driverless. Kabzan et al. use a fastSLAM algorithm and provide data of an evaluation of the algorithm [8]. Anderson and Baerveldt chose and discussed another algorithm, ORB-SLAM2 [9]. Wahlqvist implemented an EKF algorithm on a modified remote controlled car to test different motion priors with an application on a Formula Student Driverless car in mind [10]. Similarly, Brunnbauer and Bader implemented and evaluated an EKF-based localization algorithm on a 1:10 car [11]. However, no concrete comparison to a different SLAM algorithm is made in any of these references.

Outside of Formula Student, SLAM approaches are evaluated, for example, in the context of Roborace [12] or the DARPA challenge [13]. However, these settings differ quite significantly from the Formula Student application.

2 Underlying conditions

Formula Student is an international design competition for students. With self-developed and self-built race cars, the participating teams compete in various events all over the world. KA-RaceIng is the Formula Student team home to the Karlsruhe Institute of Technology and is actively and successfully participating in the Formula Student competitions, including Formula Student Driverless since its introduction in 2017. The presented work is conducted in the course of this competition. Therefore, underlying conditions are set to the corresponding rules and regulations.

2.1 Challenge

The Formula Student Driverless competition consists of various disciplines with the biggest challenge for the

SLAM algorithm being the Autocross discipline. In this time-challenge, the vehicle races on a narrow circuit marked by traffic cones that is unknown beforehand, aside from constraints such as a minimum track width of 3 m or a maximum length of 500 m. In consequence, the vehicle must map the environment using its onboard sensors to navigate around the track. The scoring is based on the lap time with points being deducted when the vehicle leaves the track or hits cones and no points awarded if it is unable to finish the lap.

2.2 Base car: KIT19d

The data set is recorded on KA-RaceIng's 2019 driverless car, the KIT19d, depicted in Figure 1.

To measure the environment, the vehicle uses two types of sensors: In the front of the car, three lidars (IBEO LUX Model 2010) deliver information about the distance of the surroundings in the form of a point cloud. Mounted on the highest point of the car, the main roll hoop, are the two cameras (Basler dart daA1600-60uc).

Additionally, each of the four motors is fitted with a hall sensor measuring its respective angular velocity. Using the fixed gearing ratio and the wheel's diameter, the wheel speeds are calculated from this measurement. An inertial measurement unit (Xsens MTi-G-710) is mounted in the center of the car. Together, these sensors are used to compute a velocity estimate of the vehicle.

2.3 Perception

The perception pipeline takes advantage of the different strengths of both lidar and camera sensors. The lidars serve as the primary information source to identify objects and their respective position relative to the vehicle, while the camera's information is used to verify and classify detected objects. We use the DBSCAN algorithm [14] to filter and cluster the lidar measurements. Clusters that represent potential cone measurements are then projected into image space. To reduce computational complexity, only areas of the image where a cone is proposed by the lidar, will be further investigated. For this, the distance of the potential cone is used to mark the region of interest in the image using a bounding box as shown in Figure 2.

Subsequently, each region of interest is cut out and fed into a small convolutional neural network. It classifies the content in one of four categories: no cone, a blue cone (left side of the track), yellow cone (right side of the track) or orange cone (start of the track).



Figure 2: Example results of KIT19d's perception pipeline. In our work we make use of the estimated cone locations.

3 SLAM

To solve the SLAM problem, two approaches, an EKF SLAM and a GraphSLAM algorithm, are discussed in this work and compared with respect to their performance. The SLAM problem is regarded in two dimensions as Formula Student tracks are assumed to be flat. To match observed and mapped landmarks, the JCBB data association algorithm [15] is used. The implementation is taken from the MRPT library [16].

3.1 Motion and measurement model

Both algorithms make use of the same motion and measurement model, which are described in the following. The position of the car within the global reference frame at time t is described by $x_{v,t}$ and $y_{v,t}$ while $\varphi_{v,t}$ describes the yaw angle of the vehicle with respect to the global x -axis. The initial position of the car is defined as $(x_{v,0}, y_{v,0}, \varphi_{v,0})^T = (0, 0, 0)^T$.

The motion model

$$\begin{aligned} x_t &= \begin{pmatrix} x_{v,t} \\ y_{v,t} \\ \varphi_{v,t} \end{pmatrix} = \mathcal{N}(0, Q_t) + \begin{pmatrix} x_{v,t-1} \\ y_{v,t-1} \\ \varphi_{v,t-1} \end{pmatrix} \\ &\quad + \begin{pmatrix} \cos(\varphi_{v,t-1})v_x\Delta t - \sin(\varphi_{v,t-1})v_y\Delta t \\ \sin(\varphi_{v,t-1})v_x\Delta t + \cos(\varphi_{v,t-1})v_y\Delta t \\ \dot{\varphi}\Delta t \end{pmatrix} \\ &= \mathcal{N}(0, Q_t) + g(u_t, x_{t-1}) \end{aligned} \quad (1)$$

estimates the next vehicle state $x_t = (x_{v,t}, y_{v,t}, \varphi_{v,t})^T$ from the odometry measurement at time t , $u_t = (v_x, v_y, \dot{\varphi})^T$, and

the previous vehicle state x_{t-1} . Here, v_x and v_y represent the measurement of the car's linear velocity and $\dot{\phi}$ describes the measured yaw rate. It is evaluated over a time span Δt , which approximates the time until a new odometry measurement is available. The motion is modeled probabilistically using a zero-mean Gaussian probability distribution with a covariance described by Q_t . The Gaussian probability distribution is chosen because it is a necessary requirement for Kalman-based filter algorithms and results have shown this approximation to be sufficient. g is called the state transition function.

The measurements received from the perception pipeline are arrays of cylindrical landmark coordinates in the vehicle coordinate frame. The j -th measurement at time t z_t^j is represented by its distance to the car r_t^j and the angle between the measurement and the local x -axis θ_t^j .

The measurement model

$$z_t^j = \begin{pmatrix} r_t^j \\ \theta_t^j \end{pmatrix} = \mathcal{N}(0, R_t) \quad (2)$$

$$+ \begin{pmatrix} \sqrt{(x_{j,g} - x_{v,t})^2 + (y_{j,g} - y_{v,t})^2} \\ \text{atan2}(y_{j,g} - y_{v,t}, x_{j,g} - x_{v,t}) - \phi_{v,t} \end{pmatrix}$$

$$= \mathcal{N}(0, R_t) + h(x_t, j, m)$$

uses the landmark position in the global reference frame $(x_{j,g}, y_{j,g})^T$ and the vehicle state at time t , x_t , to compute the measurement. The uncertainty of the measurement is modelled with a zero-mean Gaussian distribution with a covariance R_t . m is the map containing the global positions of all landmarks, j is an ID unique to each landmark, and h is the measurement function. $\text{atan2}(y, x)$ returns the angle between the vector $(x, y)^T$ and the x -axis with values ranging from $-\pi$ to π .

3.2 EKF SLAM

The EKF SLAM algorithm used in this thesis is largely based on the algorithm described in [4]. The state contains the vehicle state and the landmark positions. In the prediction step of the filter, the next vehicle state is predicted using odometry measurements and a motion model. In the update step, the vehicle state as well as all landmark positions are corrected using the measured landmark positions and a measurement model. Compared to the algorithm described in [4], a few changes have been made. The motion and measurement model described above differ to better suit the specific application. New landmarks will be initialized using the expected landmark position instead of a generic position.

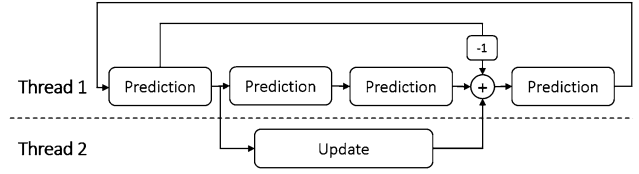


Figure 3: Architecture of the EKF SLAM implementation.

Additionally, the entire structure of the algorithm is parallelized to account for different rates of the odometry and the lidar sensors and to ensure meeting all time constraints. Opposed to a standard Kalman filter implementation, the presented algorithm works with the architecture depicted in Figure 3. The entire update step is moved onto a separate thread allowing for ongoing predictions, even if an update is currently in progress. This ensures that no information is lost, even if the update step requires a rather long time. The updated step and the predicted state are then fused as soon as the update finished processing. Additionally, in this structure, the fact that the prediction occurs about four times as often as the update step due to different sensor rates can be easily accommodated.

3.3 GraphSLAM

Opposed to a Kalman filter approach, the GraphSLAM algorithm [17] tries to capture the entire SLAM problem in a graph-based structure to find a configuration that best satisfies a global optimization problem. The GraphSLAM algorithm is implemented using the g2o library [18]. Each vehicle pose is represented by a pose node and each mapped landmark is represented by a landmark node. Consecutive pose nodes are connected by odometry edges. Landmark vertices are connected to all vehicle poses they have been measured from using landmark edges.

A new odometry input is used to generate a new pose node using the same motion model as the EKF SLAM algorithm. Additionally, the new pose node is connected with the pose node prior to it. As described in [17], the constraint linking these poses is a squared error term. The error is defined as the difference between the current vehicle pose and the predicted vehicle pose with the term being weighted using the measurement covariance. This leads to

$$(x_t - g(u_t, x_{t-1}))^T Q_t^{-1} (x_t - g(u_t, x_{t-1})). \quad (3)$$

Observations that can be matched to a mapped landmark will be linked to the vehicle pose that the measurement had been made from using a landmark edge. Analogously

to Eq. (3), the resulting constraint is

$$(z_t^j - h(x_t, j, m))^T R_t^{-1} (z_t^j - h(x_t, j, m)). \quad (4)$$

using the difference between the actual and the expected measurement as the error term. For landmarks that have never been seen before, a new landmark node are created and connected using a landmark edge just as the matched landmarks above.

Finally, a graph is built forming a cost function

$$\begin{aligned} J = & x_0^T \Omega_0 x_0 \\ & + \sum_t (x_t - g(u_t, x_{t-1}))^T Q_t^{-1} (x_t - g(u_t, x_{t-1})) \\ & + \sum_t \sum_j (z_t^j - h(x_t, j, m))^T R_t^{-1} (z_t^j - h(x_t, j, m)) \end{aligned} \quad (5)$$

from the sum of all constraints. The first term, called “anchoring constraint”, uses an initial vehicle state x_0 and an initial information matrix Ω_0 (i. e. inverse covariance matrix) to fix the problem to a global coordinate frame. As described above, in our case the initial vehicle position is arbitrarily chosen to be $x_0 = (0, 0, 0)^T$. The cost function is then minimized using the Levenberg-Marquardt optimization algorithm [19]. Because the optimization is a computationally expensive process, the entire algorithm is parallelized as visualized in Figure 4.

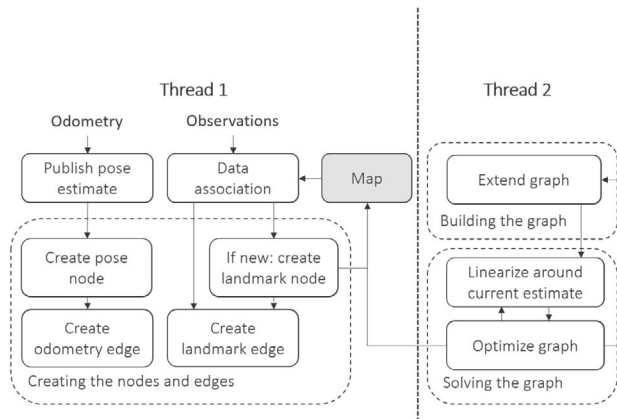


Figure 4: Architecture of the GraphSLAM implementation.

The key idea is to move the optimization onto a second thread so that it can run in the background, while the algorithm is still collecting all incoming data to make sure no information is lost. Thread 1 will create all the necessary nodes and edges whenever new odometry or landmark observations are available. However, instead of building the graph directly, the data is stored in a buffer. Additionally,

thread 1 will always publish a vehicle pose estimate in real-time.

Whenever Thread 2 is ready for another optimization, it will then take the data from the buffer to extend the already existing graph with the newly acquired information and commence the next optimization. This optimization will then reduce the drift in the short term vehicle pose estimate made by thread 1 to increase the overall accuracy.

4 Evaluation

To measure the performance of the algorithms presented in this paper, they are analyzed with respect to their accuracy and their efficiency. For the latter, the time taken to process incoming data (and in the case of GraphSLAM, the time taken for the optimization itself) is measured to ensure that all messages are processed in time. Here, the time spans available to process odometry and perception measurements are 10 ms and 40 ms, respectively. Additionally, the CPU usage is used as a measure of efficiency. This is an important benchmark because in practice, there are a number of other algorithms running at the same time and the computational resources are limited by external factors.

Measuring the accuracy is a difficult problem in itself because it is impossible to say which of the algorithms provides the better estimate without having a reference. Such a reference was acquired in the form of the ground truth map depicted in Figure 1. The positions of all cones have been marked on the ground and were measured using a high-precision DGPS device. The DGPS measurements have an error below 1 cm. Considering the extent of the marking itself, it is safe to assume that the error of the measured cone positions is below 5 cm.

An EKF SLAM algorithm that was implemented by the KA-RaceIng team of 2019 is used as a baseline. In the following, it is referred to as “EKF SLAM 2019” and works very similarly to the algorithm presented in this work, referred to as “EKF SLAM 2020”. However, the map is never corrected and the algorithm does not use the parallelized structure presented earlier.

4.1 Accuracy

Since the total number of cones on the track is known, looking at the total number of mapped cones can give an insight to whether the algorithms filter too strictly or whether they do not match cones that do in fact correspond.

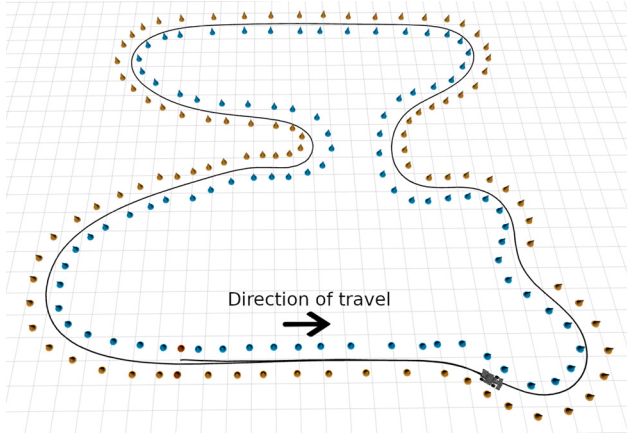


Figure 5: Resulting map and path estimate using the presented GraphSLAM algorithm.

Figure 6(a) shows the results for one lap of Autocross. The number of cones are very similar for EKF SLAM 2020 and GraphSLAM but EKF SLAM 2019 tends to map too many cones, which is likely due to the lower accuracy of the algorithm which results in failed associations.

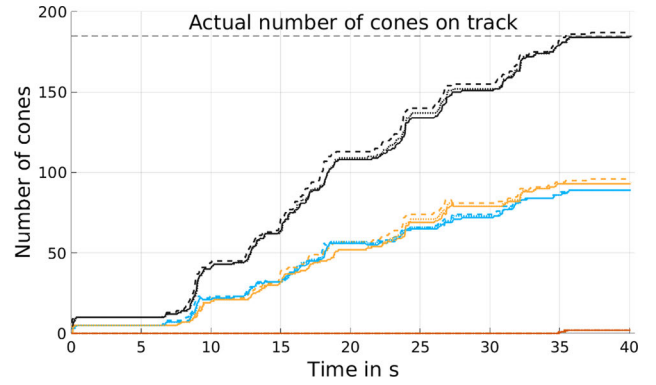
When comparing the SLAM-generated map to the ground truth map, an interesting measure is the number of matches found on both maps. While some missing correspondences may be rooted in errors in the data set, a match can usually not be found simply due to the fact that the SLAM estimate of the cone's position is too far off.

The matching ratio describes the ratio of matched cones n_{matched} and the number of detected cones n_{detected} , per time step:

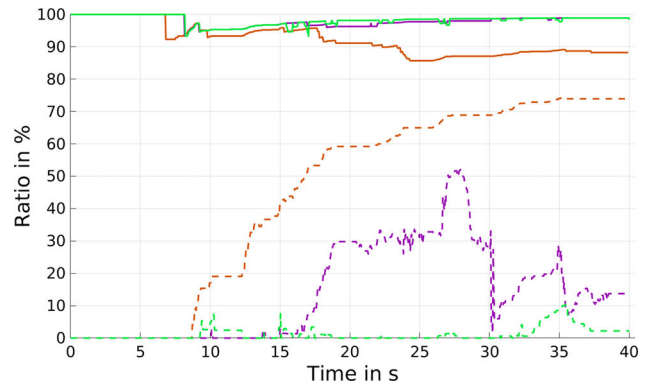
$$m = \frac{n_{\text{matched}}}{n_{\text{detected}}}. \quad (6)$$

Due to minor errors in the data set, the matching ratio shown in Figure 6(b) should ideally be close to 99 %. This is the case for both EKF SLAM 2020 and GraphSLAM. Both algorithms drop to values between 93 % and 97 % in the middle section but do recover from this. EKF SLAM 2019 drops to about 86 % in the middle section and finishes the lap with less than 90 % of matchings made.

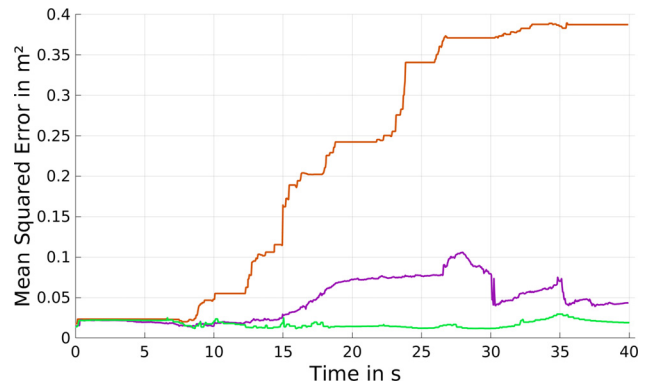
In practice, another measure is of interest: The percentage of mapped cones with a positioning error above a threshold of 30 cm. Positioning errors below this threshold are less of an issue because the car will have to plan its trajectory with a safety margin anyway. However, considering the narrowness of the race tracks, a positioning error larger than that will force the car off the racing line in order not to hit any cones ultimately resulting in increased lap times.



(a) Number of blue, yellow, orange and total cones mapped by EKF SLAM 2019 (large dashes), EKF SLAM 2020 (continuous), and GraphSLAM (small dashes).



(b) Matching ratio (continuous) and ratio of matched cones with a positioning error above a threshold of 30 cm (dashed) by EKF SLAM 2019 (orange), EKF SLAM 2020 (purple), GraphSLAM (green). The matching ratio is calculated by dividing the number of matched cones by the number of cones on the SLAM-generated map.



(c) Mean squared error of the positions of all mapped cones where a corresponding cone was found on the reference map, per time step, for EKF SLAM 2019 (orange), EKF SLAM 2020 (purple), and GraphSLAM (green).

Figure 6: Accuracy benchmarks.

As shown in Figure 6(b), EKF SLAM 2019 has most of the map, or about 74 %, above the threshold. The graph of EKF SLAM 2020 also rises to values of up to 53 %, however, at around 30 s and 35 s, big drops of the graph can be seen that can be explained by loop closures. This can not be observed in the graph of EKF SLAM 2019 because this algorithm never corrects the cone positions and therefore cannot reduce the error.

GraphSLAM performs a lot better, with a maximum of 10.5 % of cones above the positioning error threshold and roughly 2 % at the end of the run.

The mean squared error in Figure 6(c) is interesting because it is a very absolute and precise measure of the accuracy over the course of the lap.

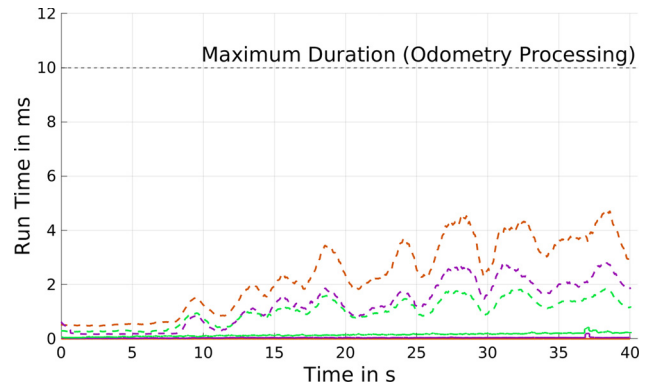
Like above, GraphSLAM is the most accurate algorithm followed by EKF SLAM 2020 with a big improvement over EKF SLAM 2019. The mean squared error of the latter increases almost continuously to a value of 0.388 m^2 . Again, this is due to the lack of corrections made. The increasing error reflects the drift over the course of the lap. EKF SLAM 2020 and GraphSLAM finish the lap with much better values of 0.044 m^2 and 0.019 m^2 , respectively. GraphSLAM's superior accuracy is also shown by the fact that it is at constantly low values, whereas EKF SLAM 2020 increases to significantly higher values over the course of the lap that are only corrected after loop closure.

4.2 Efficiency

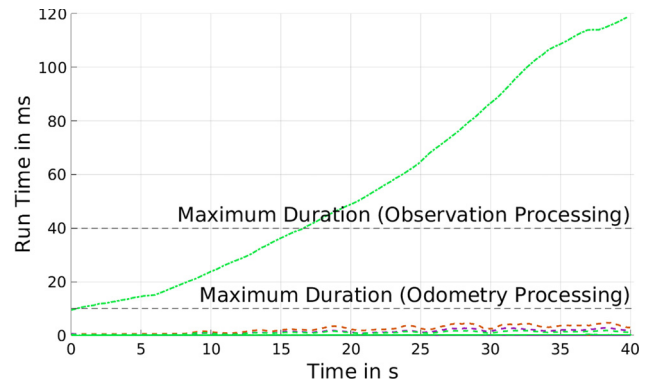
Incoming odometry data and observations trigger a callback in both algorithms. The time taken to compute the callback is measured and plotted in Figure 7(a).

Odometry measurements are processed in under 0.5 ms, which is why they are barely visible in the plot. GraphSLAM takes the longest for this, but even the highest peak at around 37 s is easily within the available time span of 10 ms. Another interesting observation in this data is that the time taken by GraphSLAM to process odometry data increases over the course of the lap. This can be explained by the adjustment of short term estimates. As discussed further below, the duration of an optimization will increase over time, meaning that more short-term pose estimates are being made based on the velocity estimate that need to be corrected after an optimization is complete. Still, the odometry data is easily processed quick enough.

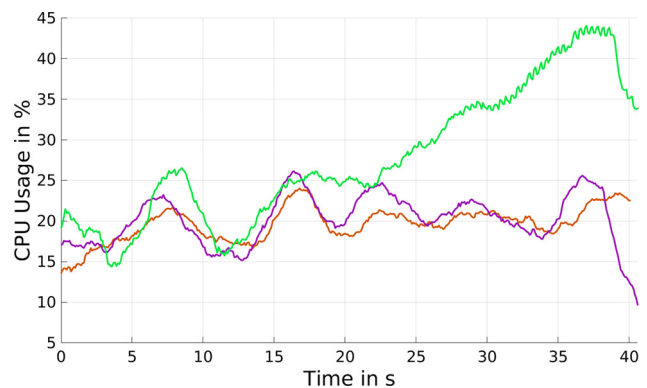
The dashed lines represent the times taken by the algorithms to process observations. First of all, it should be noted that processing an observation is allowed to take up to 40 ms. If it takes longer, new observations cannot be



(a) Time taken to process new odometry data (continuous) and observations (dashed) by EKF SLAM 2019 (orange), EKF SLAM 2020 (purple), GraphSLAM (green). The run times are filtered using a moving average filter. The maximum time an odometry processing may take before information is lost is marked in the plot.



(b) Same as (a) but including GraphSLAM optimization run times (dot-dashed).



(c) CPU usage of EKF SLAM 2019 (orange), EKF SLAM 2020 (purple), GraphSLAM (green) during one lap of Autocross.

Figure 7: Efficiency Benchmarks.

processed in time and information is lost. All algorithms perform their computations well within that time limit. The first thing that stands out when looking at the three

graphs are the local peaks. They can be explained with the number of cones seen in a specific time step as the algorithms take longer the more measurements are available.

The next observation is an unexpected one: EKF SLAM 2019 actually takes longer than EKF SLAM 2020. This should not be the case because it only corrects the vehicle pose and does not improve the map estimate. This effect may be explained by the lower accuracy which makes the process of data association a lot harder and by an overall improved code quality.

When comparing EKF SLAM 2020 and GraphSLAM, it can be seen that they take a similar amount of time in the beginning with an increasing offset towards the end of the run, which can be explained by the EKF's computational complexity. Because the map is part of the state and built over the course of the lap, the number of states and with that the complexity of the EKF SLAM algorithm increases continuously.

However, the comparison between the observation processing of EKF SLAM 2020 and GraphSLAM is not a fair one. EKF SLAM 2020 will correct the map and the vehicle pose in this step, whereas GraphSLAM merely prepares the optimization, which is not shown in Figure 7(a) but is included in Figure 7(b). Due to the number of constraints increasing with time, the computational cost of the optimization increases over the course of a lap. As can be seen in the plot, this time dwarfs the run times of the odometry and observation processing. It should be noted that unlike processing the input data, optimizing the graph does not have a fixed time limit. Optimizing with a higher frequency will surely increase the short-term accuracy but if an optimization takes longer, no information is lost because collecting data and optimizing the graph is done on separate threads.

The accuracy presented in Section 4.1 was achieved with one optimization for every ten sets of observations, i. e. every 400 ms. This means that the optimization could even be performed three times as often without reaching that limit in this data set.

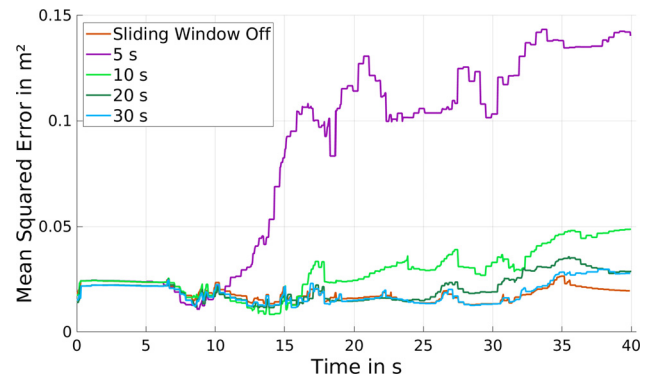
Finally, the usage of computational resources of the algorithms are analyzed in Figure 7(c). When comparing the CPU loads of EKF SLAM 2019 and EKF SLAM 2020, a similar observation as above can be made: EKF SLAM 2019 should be more efficient but it is not, likely due to the lower accuracy and reduced overhead.

The CPU usage of GraphSLAM turns out to be a larger issue than the run times. With an increasing number of constraints, the optimization becomes the dominant process in terms of CPU load after around 20 s. At the end of the run, GraphSLAM takes up close to 45 % of the CPU,

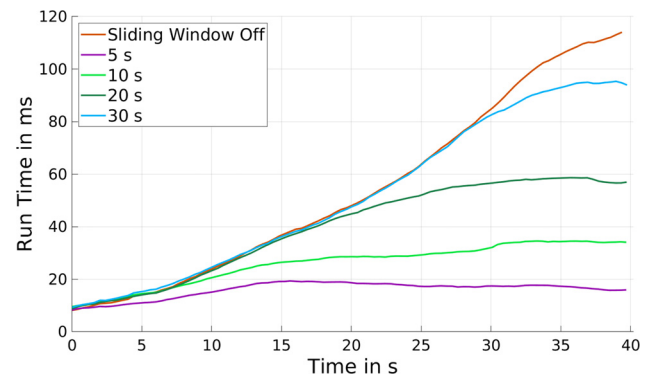
almost twice the value of either of the EKF SLAM algorithms.

4.3 Sliding window

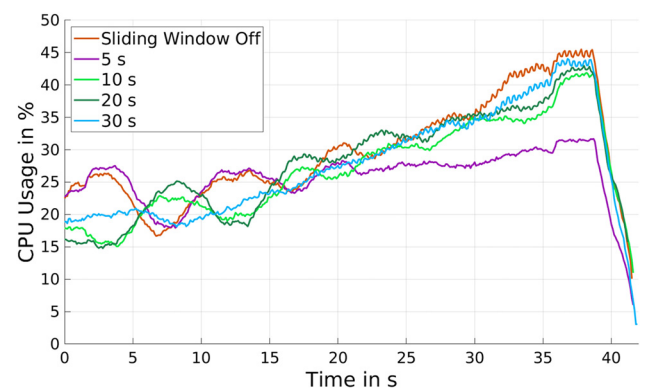
In the case of GraphSLAM, the accuracy and the efficiency of the algorithm can be weighted against each other using



(a) Mean squared error



(b) Run times



(c) CPU Usage

Figure 8: GraphSLAM: Performance impact of sliding windows of different sizes.

Table 1: Summary of the results of the performance analysis.

			EKF-SLAM 2019	EKF-SLAM 2020	Graph-SLAM
Accuracy	Number of Cones [–]	Final	187	184	184
	Matching Ratio in %	Min	85.71	93.33	93.26
		Final	88.24	98.91	98.91
	Cones Above Thres. in %	Max	74.39	53.74	10.50
		Final	73.94	14.84	2.20
	MSE in m ²	Max	.3894	.1060	.0296
		Final	.3875	.0436	.0189
Efficiency	Run Time in ms	Odom.	.01	.2	.43
		Obser.	4.67	2.83	1.84
		Optim.	–	–	118.6
	CPU Usage in %	Max	24.03	26.14	43.93

a sliding window which limits the amount of constraints considered when performing an optimization. To determine the impact a sliding window has on the accuracy of the algorithm, different window sizes are compared to a run where the sliding window is turned off using the measures presented above. The window sizes are chosen such that they include roughly 5 s, 10 s, 20 s, and 30 s worth of constraints. In comparison, the lap time of the track depicted in Fig. 5 is approximately 30 s.

The mean squared error of these windows are depicted in Figure 8(a). The results confirm the expected: The larger the sliding window, the higher the accuracy with the best results achieved without a sliding window. It is notable that the accuracy does not grow linearly with the size of the sliding window. Very small windows will have a poor performance whereas increasing the size of large windows is barely noticeable.

The benefit of the sliding window approach can be seen in Figure 8(b). As expected, the run times of the optimization scales linearly in the number of edges, i. e. the size of the window. Choosing the right windows size or whether a sliding window should be used at all should therefore be a compromise between the required accuracy and the available resources.

Because the optimization is the dominant process only towards the end of a run, the effect a sliding window has on the CPU usage is lower as can be seen in Figure 8(c). The only window size where the CPU usage is decreased significantly is the smallest one (5 s). All other windows reach a maximum CPU load of 40 % to 45 %.

In conclusion: In terms of CPU usage, a sliding window does not make a big difference in the short time span of an Autocross run.

Table 1 summarizes the results of all benchmarks presented above.

5 Conclusion

In this work, we design and analyze the implementation of a filter-based and a smoothing-based approach to the SLAM problem for the application on a race car. The underlying conditions are set by the Formula Student Driverless competition. We evaluate the algorithms in terms of their performance using a high-precision ground truth map.

While both algorithms manage to solve the SLAM problem, the GraphSLAM algorithm outperforms the EKF SLAM algorithm in terms of accuracy while keeping the required time constraints. However, GraphSLAM drains the computational resources very quickly, even though a sliding window can reduce this issue.

In contrast, EKF SLAM operates more efficiently due to the limited size of the map. With the improvements made over the previously implemented EKF SLAM algorithm, the algorithm presented in this work still delivers a reasonable accuracy while keeping the CPU usage at a lower level. Therefore, if computational resources are an issue, EKF SLAM is a valid alternative.

Acknowledgment: The authors want to thank the KA-RaceIng Team of 2020 and all its supporters for the time, effort and resources that were spent to enable this project and lead to KA-RaceIng's victory in the Formula Student Online (FSO) competition in 2020.

References

1. Formula E. URL: <https://www.fiaformulae.com/> (accessed 26.02.2021).
2. Roborace. URL: <https://roborace.com/> (accessed 26.02.2021).
3. I. of Mechanical Engineers. *History of Formula Student*. URL: <https://www.imeche.org/events/formula-student/about->

formula-student/history-of-formula-student/ (accessed 22.05.2020).

4. S. Thrun, W. Burgard and D. Fox. *Probabilistic Robotics*. Cambridge, Mass.: MIT Press, 2005. ISBN: 0262201623, 9780262201629.
5. M. Zeilinger et al. “Design of an autonomous race car for the Formula Student Driverless (FSD)” (May 2017).
6. M. Valls et al. “Design of an autonomous racecar: Perception, state estimation and system integration” (Apr. 2018).
7. S. Nekkah et al. “The autonomous racing software stack of the KIT19d”. *arXiv e-prints*, arXiv:2010.02828 [cs.RO] (Oct. 2020).
8. J. Kabzan et al. “AMZ driverless: The full autonomous racing system” (May 2019).
9. M. Anderson and M. Baerveldt. “Simultaneous localization and mapping for vehicles using ORB-SLAM2”. M. Sc. Thesis. Gothenburg, Sweden: Chalmers University of Technology, May 2018.
10. K. Wahlqvist. “A comparison of motion priors for EKF-SLAM in autonomous race cars”. M. Sc. Thesis. Stockholm, Sweden: KTH, School of Electrical Engineering and Computer Science (EECS), July 2019.
11. A. Brunnbauer and M. Bader. “Traffic cone based selflocalization on a 1:10 race car” (2019).
12. F. Nobis et al. “Autonomous racing: A comparison of SLAM algorithms for large scale outdoor environments”. In: *ICVARS '19*, Feb. 2019, pp. 82–89. DOI: 10.1145/3332305.3332319.
13. L. B. Cremean et al. “Alice: An information-rich autonomous vehicle for high-speed desert navigation”. In: *The 2005 DARPA Grand Challenge: The Great Robot Race*. M. Buehler, K. Iagnemma and S. Singh (eds.). Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 437–482. ISBN: 978-3-540-73429-1. DOI: 10.1007/978-3-540-73429-1_14.
14. M. Ester et al. “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise”. AAAI Press, 1996, pp. 226–231.
15. J. Neira and J. D. Tardos. “Data association in stochastic mapping using the joint compatibility test”. *IEEE Transactions on Robotics and Automation* 17(6) (2001), pp. 890–897.
16. J. L. Blanco. “Development of scientific applications with the Mobile Robot Programming Toolkit” (Jan. 2008).
17. G. Grisetti et al. “A tutorial on graph-based SLAM”. *IEEE Intelligent Transportation Systems Magazine* 2(4) (2010), pp. 31–43. DOI: 10.1109/MITS.2010.939925.
18. R. Kummerle et al. “G2o: A general framework for graph optimization”. In: *2011 IEEE International Conference on Robotics and Automation* (2011), pp. 3607–3613.
19. D. W. Marquardt. “An algorithm for least-squares estimation of nonlinear parameters”. *Journal of the Society for Industrial and Applied Mathematics* 11(2) (1963), pp. 431–441. ISSN: 0368-4245.

Bionotes



Nick Le Large

Institut für Mess- und Regelungstechnik (MRT), Karlsruher Institut für Technologie (KIT), Engler-Bunte-Ring 21, 76131 Karlsruhe, Germany
nick.lelarge@kit.edu

Nick Le Large received a Master's degree in mechanical engineering from the Karlsruhe Institute of Technology, where he is currently pursuing a Ph. D. degree at the Institute of Measurement and Control Systems. In 2019, he joined KA-RaceIng to participate at the Formula Student Driverless challenge. He was responsible for the localisation and mapping system of KA-RaceIng's Formula Student Driverless race car of 2020.



Frank Bieder

FZI Forschungszentrum Informatik, Haid-und-Neu-Straße 10-14, 76131 Karlsruhe, Germany
bieder@fzi.de

Frank Bieder received a Master's degree in electrical engineering and information technology from the Karlsruhe Institute of Technology, where he is currently pursuing a Ph. D. degree in computer vision and machine learning at the Institute of Measurement and Control Systems. Since 2019, he is a research scientist at the Mobile Perception Systems Department, FZI Research Center for Information Technology and a Doctoral Researcher at the Karlsruhe School of Optics & Photonics (KSOP).



Martin Lauer

Institut für Mess- und Regelungstechnik (MRT), Karlsruher Institut für Technologie (KIT), Engler-Bunte-Ring 21, 76131 Karlsruhe, Germany
martin.lauer@kit.edu

Martin Lauer received the Diploma degree in computer science from the Karlsruhe Institute of Technology and the Ph. D. degree in computer science from Osnabrück University in 2004. He was a Post-Doctoral Researcher with Osnabrück University in the areas of machine learning and autonomous robots. Since 2008, he has been heading a research group at the Karlsruhe Institute of Technology. His main research interests are in the areas of machine vision, autonomous vehicles, and machine learning.